

데이터베이스 HW#1 202412999 양현성

1. 서론

본 레포트는 MySQL 환경에서의 주어진 테이블 환경 내에서 특정한 문제를 해결하기 위한 쿼리 설계와 해당 쿼리 설계의 결과 검증에 관한 내용을 담는다. 이 환경에서는 강의에서 진행된 SQL 테이블 환경 혹은 튜플 내용과 상이하여 결과값이 다를 수 있음을 밝힌다. 다만, 3번 항목의 경우 동일하다.

2. 선제시 테이블 쿼리 작성

2-1.

튜플에서 데이터를 추출하기 위해서는 email 컬럼 중 LIKE 문자열 탐색과 join_date 컬럼의 2026년 1월 1일 이후 데이터를 제한조건으로 설정할 필요가 있다.

```
select user_name, join_date from Users where email like '%email.com%' and join_date > '2026-1-1' order by join_date asc;
```

Result

```
+-----+-----+
| user_name | join_date |
+-----+-----+
| 김표순   | 2026-01-15 |
| 이표돌   | 2026-02-10 |
+-----+-----+
2 rows in set (0.01 sec)
```

2-2.

튜플에서 데이터를 추출하기 위해서 장르 컬럼이 액션 혹은 스릴러여야 하기에 조건문을 통하여 제한을 걸고, rating 또한 9.0 이상으로 제한을 걸게 한다.

```
select title, genre, rating from Contents where (genre = 'Action' or genre = 'Thriller') and rating > 9.0;
```

Result

```
+-----+-----+-----+
| title          | genre  | rating |
+-----+-----+-----+
| 오징어 게임 시즌2 | Thriller | 9.2 |
| 무빙           | Action  | 9.5 |
| 어벤저스: 엔드게임 | Action  | 9.5 |
+-----+-----+-----+
3 rows in set (0.01 sec)
```

2-3.

레이팅 컬럼에서 평균 총합을 얻어내기 위해 **AVG()** 함수를 사용하였고, 조건은 작품이 2개 이상이어야 하기 때문에 장르 그룹으로 묶인 튜플들의 갯수를 **COUNT()**를 통하여 센 뒤 필터링한다.

```
select genre, avg(rating) as avg_rate from Contents group by genre having count(genre) >= 2;
```

Result

```
+-----+-----+
| genre | avg_rate |
+-----+-----+
| Action | 9.50000 |
+-----+-----+
1 row in set (0.00 sec)
```

2-4.

EXCEPT 문을 활용하여 기존 튜플들에서 튜플을 제거하기 때문에, 기존 튜플 집합은 테이블에서 모든 튜플을 불러온 뒤, **Premium** 유저만을 골라 튜플에서 제거할수 있도록 **Premium** 유저만을 열거하는 튜플 집합을 만든다.

```
select user_id from Users except select user_id from Users where membership_type = 'Premium';
```

Result

```
+-----+
| user_id |
+-----+
| bbb     |
+-----+
1 row in set (0.00 sec)
```

2-5.

멤버십 내에서 유저별로 묶어야 하기 때문에 멤버십 타임 컬럼을 기준으로 튜플들을 그룹화하고, 그룹화된 튜플에서 **SUM()**을 이용해 값을 합산한 뒤 정렬한다.

```
select membership_type, sum(fee) as Total_Revenue from Users group by membership_type order by Total_Revenue desc;
```

Result

```
+-----+-----+
| membership_type | Total_Revenue |
+-----+-----+
| Premium        | 34000         |
+-----+-----+
```

3. 특정 시나리오에 따른 테이블 및 쿼리 설계

3-1.

영화 테이블에서 중복되지 않아야 하는 데이터는 영화 튜플의 영화 ID이므로, ID를 int인 Primary Key로 설정하게 한다.

```
create table movies (  
  -> movie_id int primary key,  
  -> title varchar(100),  
  -> genre varchar(30),  
  -> director varchar(50),  
  -> release_year int,  
  -> ticket_price int);
```

3-2.

사용자 테이블에서 중복되지 않아야 하는 데이터는 사용자 튜플의 사용자 ID(혹은 유저네임)이므로, ID를 varchar인 Primary Key로 설정하게 한다.

```
create table users(  
  -> viewer_id varchar(20) primary key,  
  -> viewer_name varchar(50),  
  -> city varchar(30),  
  -> grade varchar(20));
```

3-3.

예약 테이블에서 중복되지 않아야 하는 데이터는 예약 튜플의 예약 ID이므로, ID를 int인 Primary Key로 설정하게 한다. 또한 사용자 ID와 영화 ID는 서로의 원본 테이블에 존재해야 하는 값인지 확인할 필요가 있으므로 이 둘을 Foreign Key로 설정한다.

```
create table reservation(  
  -> reserve_id int primary key,  
  -> viewer_id varchar(20),  
  -> movie_id int,  
  -> reserve_date date,  
  -> pos int,  
  -> foreign key(viewer_id) references users(viewer_id),  
  -> foreign key(movie_id) references movies(movie_id));
```

3-4.

장르의 갯수를 세어야 하기 때문에, 장르 컬럼을 기준으로 튜플들을 그룹화한 뒤, COUNT() 함수를 이용하여 장르 컬럼을 기준으로 갯수를 센다. 그 후 COUNT된 값을 중심으로 내림차순 정렬을 하여 장르에서 발매된 영화가 많은 순으로 반환되게 한다.

```
select genre, count(genre) as counts from movies group by genre order by counts desc;
```

Result

```
+-----+-----+
| genre | counts |
+-----+-----+
| Action |    2 |
| Drama  |    2 |
| Sci-Fi |    1 |
| Thriller |    1 |
+-----+-----+
4 rows in set (0.00 sec)
```

3-5.

영화 이름, 좌석 수, 예약자 이름을 불러와야 하기 때문에 먼저 세 테이블의 Alias를 설정한 뒤 불러온다. 그 후 좌석 수가 2개 이상인지 확인하고, 예약자가 고른 영화만 표기해야 하기에 예약정보 튜플에서 영화의 ID가 맞는지, 그리고 신청자가 맞는지 확인하게 한다.

```
select u.viewer_name, m.title, r.pos as seat_count from users as u, movies as m, reservation as r where r.pos >= 2 and r.movie_id = m.movie_id and r.viewer_id = u.viewer_id;
```

```
+-----+-----+-----+
| viewer_name | title          | seat_count |
+-----+-----+-----+
| Kim Jisu   | Urban Night   |    2 |
| Park Minji | Galaxy Mission |    3 |
| Choi Hyerin | Spring Letter |    2 |
| Han Junseo | Urban Night   |    4 |
| Lee Doyun  | Fast Track    |    2 |
+-----+-----+-----+
5 rows in set (0.00 sec)
```

4. AI를 사용한 테이블 및 쿼리 설계 및 풀이

Google Gemini Pro 3.1을 사용하여 Subquery를 제외한 범위에서 2개의 테이블을 활용하는 5문제를 부여받았다. SQLD 수준의 난이도, Subquery를 제외하는 조건을 지키기 위하여 강의자료를 모두 업로드하였으며, 해당 강의자료를 통하여 문제를 생성받았으며, 투입한 프롬프트의 전문은 다음과 같다.

이 자료들은 나의 대학 **SQL** 데이터베이스 강의 자료야. 이 강의 자료 내 내용을 모두 분석하고, 이 강의 자료가 다루는 내용과 범위를 자세하게 파악한 뒤, 사용자가 직접 쿼리문을 지어내서 풀 수 있도록 **2개** 이상의 테이블 구조와 적절한 양의 데이터 튜플들을 만들어내고, 이에 관한 **5문제를** 만들어 줘.

해당 프롬프트를 통해 생성받은 시나리오에 대한 요약은 다음과 같다.

‘국내 프로야구 구단 및 선수 관리 시스템’을 주제로 선정하였다. 이 시스템은 구단 정보(**Teams**)와 해당 구단에 소속된 선수 정보(**Players**) 간의 **1:N** 관계를 기본으로 하며, 선수들의 포지션, 연봉, 타율 등의 데이터를 포함하여 실제 현업에서 발생할 수 있는 데이터 추출 상황을 가 정했다.

【 팀 테이블 】

team_id (PK)	team_name	home_city	founded_year
T01	타이거즈	Gwangju	1982
T02	트윈스	Seoul	1990
T03	베어스	Seoul	1982

【 플레이어 테이블 】

player_id (PK)	player_name	team_id (FK)	position	salary	batting_avg
----------------	-------------	--------------	----------	--------	-------------

101	김도영	T01	Infielder	15000	0.345
102	양현종	T01	Pitcher	100000	NULL
103	오지환	T02	Infielder	60000	0.280

4-1.

Players 테이블에서 타율(**batting_avg**)이 입력되지 않은(NULL인) 선수들의 이름(**player_name**)과 연봉(**salary**)을 조회하시오. 단, 연봉이 높은 선수부터 내림차순으로 정렬하여 출력하시오.

데이터를 출력하기 위해 이름과 연봉 컬럼을 선택하고, 평균값이 NULL인 튜플만 선택하도록 조건을 설정한다.

```
select player_name, salary from Players where batting_avg is null order by salary desc;
```

Result

```
+-----+-----+
| player_name | salary |
+-----+-----+
| 양현종      | 100000 |
| 광빈       | 25000  |
+-----+-----+
2 rows in set (0.00 sec)
```

4-2.

Players 테이블에서 포지션(**position**)이 'Infielder' 이면서, 이름(**player_name**)에 '지'라는 글자가 포함된 선수의 모든 정보를 조회하는 SQL문을 작성하시오.

모든 정보를 포함해야 하기에 모든 컬럼을 선택하고, 위치가 인필더, 그리고 이름에서 LIKE 기능을 이용하여 '지'를 포함하는지 검색한다.

```
select * from Players where position = 'Infielder' and player_name like '%지%';
```

Result

```
+-----+-----+-----+-----+-----+-----+
| player_id | player_name | team_id | position | salary | batting_avg |
+-----+-----+-----+-----+-----+-----+
| 101       | 김도영      | T01     | Infielder | 15000  | 0.345       |
| 102       | 양현종      | T01     | Pitcher   | 100000 | NULL        |
| 103       | 오지환      | T02     | Infielder | 60000  | 0.280       |
```

```
| 103 | 오지환 | T02 | Infielder | 60000 | 0.280 |
+-----+-----+-----+-----+-----+
1 row in set (0.01 sec)
```

4-3.

Players 테이블을 이용하여, 소속 팀(**team_id**)별로 선수들의 평균 연봉을 구하고자 한다. 단, 소속된 선수가 2명 이상인 팀의 **team_id**와 그 팀의 평균 연봉(컬럼명: **avg_salary**)을 조회하시오.

팀을 먼저 묶어 수를 세어야 하기 때문에 **GROUP BY**로 묶은 뒤, **COUNT()**를 사용하여 튜플의 갯수를 센다. 그리고 그룹화된 컬럼의 평균을 내기 위해 **AVG()**를 사용한다.

```
select team_id, avg(salary) as avg_salary from Players group by team_id having count(*) >= 2;
```

Result

```
+-----+-----+
| team_id | avg_salary |
+-----+-----+
| T01     | 57500.0000 |
| T02     | 55000.0000 |
| T03     | 52500.0000 |
+-----+-----+
3 rows in set (0.01 sec)
```

4-4.

Teams 테이블과 **Players** 테이블을 이용하여, 연고지(**home_city**)가 'Seoul'인 구단에 소속된 선수들의 이름(**player_name**), 포지션(**position**), 그리고 소속 구단 이름(**team_name**)을 조회하는 **SQL**문을 작성하시오.

이름, 위치, 팀 이름을 얻어야 하기 때문에 정보를 담는 플레이어 테이블과 팀 테이블을 동시에 불러온 뒤, 팀의 연고지가 서울인지 체크하게 하고, 현재 확인중인 플레이어가 팀에 속해있는지 동시에 검사하게 한다.

```
select p.player_name, p.position, t.team_name from Players as p, Teams as t where t.home_city = 'Seoul' and p.team_id = t.team_id;
```

Result

```
+-----+-----+-----+
| player_name | position | team_name |
+-----+-----+-----+
| 오지환      | Infielder | 트윈스    |
| 홍창기      | Infielder | 트윈스    |
| 양의지      | Catcher   | 베어스    |
```

```
| 광빈      | Pitcher | 베어스  |
+-----+-----+-----+
4 rows in set (0.01 sec)
```

4-5.

Players 테이블에서 연봉(**salary**)이 20,000 이상 60,000 이하인 타자(타율 **batting_avg**가 NULL이 아닌 선수)들의 이름과, 연봉을 12로 나눈 월급(컬럼명: **monthly_salary**)을 조회하시오.

```
select salary/12 as monthly_salary from Players where (salary >= 20000 or salary <= 60000)
and batting_avg is not null;
```

연봉을 12로 나눈 컬럼을 월급 컬럼으로 **Alias** 설정한 뒤, 20000과 60000달러 사이의 제한을 걸고, 타자만을 얻기 위해 타자 기록이 없는 튜플은 제외하게 한다.

Result

```
+-----+
| monthly_salary |
+-----+
| 1250.0000 |
| 5000.0000 |
| 4166.6667 |
| 6666.6667 |
+-----+
4 rows in set (0.00 sec)
```

5. 생성형 AI 사용에 대한 상세 내용

해당 과제의 진행을 통하여, 쿼리의 결과를 위한 적절한 컬럼 선택과 제한조건 설정, 쿼리 제한조건 검사 혹은 데이터의 정리를 위한 **GROUP BY**의 이용 방법, 그리고 **SQL**의 타입 시스템 등에 대한 상세한 지식을 얻을 수 있는 기회가 되었다.

6. 생성형 AI 사용에 대한 상세 내용

2번, 3번 항목에 대해서는 생성형 **AI**를 일절 사용하지 않았고, 풀이는 오직 작성자의 순수한 쿼리 작성으로 이루어졌다. 4번 항목에 대해선 테이블의 컨셉트, 출제 의도, 5개 문제의 출제, 테이블 튜플 데이터 등을 생성받았으며, 풀이는 도움을 받지 않았다.